

Assignment 1: Observability

Biometric Attendance Tracking System

*A Simulated Multimodal Biometric Attendance Platform with Full Metrics and Logging
Observability*

Enterprise Software Development, Fall 2026

Habib University

Submitted by: Talal Ahmed Shaikh

AI-assisted development (see Credits, Appendix)

Part A — Project Overview

Problem Statement

Manual attendance tracking — sign-in sheets, swipe cards, or spreadsheets — is slow, easy to falsify (buddy punching), and provides no real-time visibility into who is on-site at any given moment. Organizations need a fast, low-friction way to record attendance that is also hard to spoof.

Intended Users

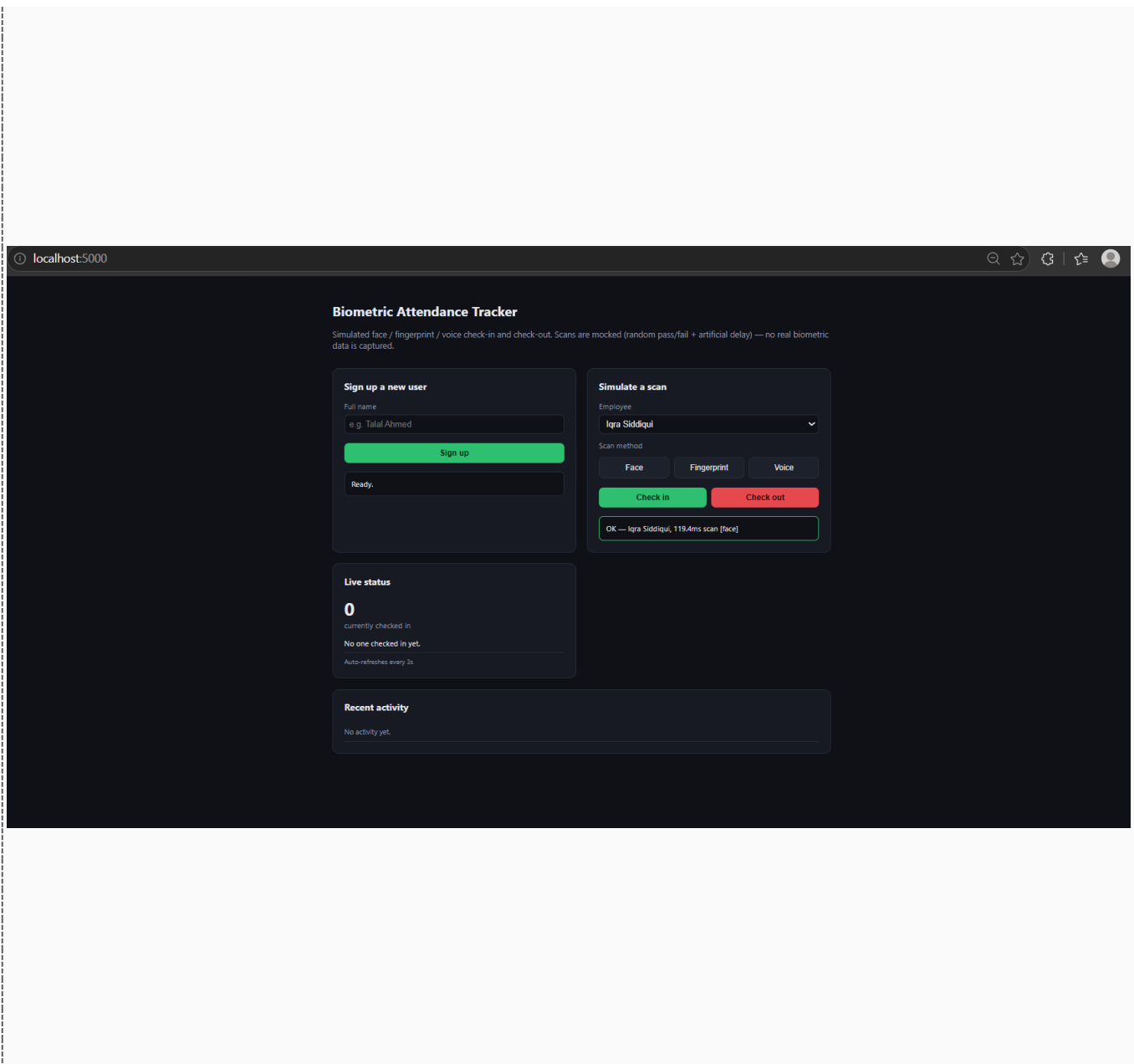
- Employees who check in and check out for each shift, using one of three biometric methods.
- System administrators who monitor attendance patterns, system health, and infrastructure performance.

Solution

A simulated multimodal biometric attendance system supporting face, fingerprint, and voice "scans". Each scan is fully mocked (no real biometric capture, model, or personal data is involved) — a scan has a configurable random chance of failing and an artificial processing delay, standing in for what a real matching algorithm's latency and error rate would look like. Employees can check in, check out, and new employees can self-enroll ("sign up"). The system is instrumented end-to-end for observability: every scan attempt, check-in, check-out, and signup produces both a Prometheus metric update and a structured JSON log line.

What Works

- A web interface (localhost:5000) to pick an employee, pick a scan method, and check in/out.
- A self-service sign-up form that enrolls a new user at runtime.
- A live status panel showing who is currently checked in, on-time vs. late, and a recent-activity feed.
- A REST API (/api/checkin, /api/checkout, /api/signup, /api/status, /api/users, /metrics) usable directly via curl or a script.
- Full metrics (Prometheus + Grafana) and logging (Filebeat + Elasticsearch + Kibana) pipelines, detailed in Parts B and C.



Screenshot 1: The frontend web interface of the simulated biometric attendance application at localhost:5000, showing the scan controls, sign-up form, and live status panel.

Part B — Metrics

Metrics are collected with the Python `prometheus_client` library and exposed at the `/metrics` endpoint, which Prometheus scrapes on a 5-second interval and stores in its Time Series Database (TSDB). Grafana queries Prometheus via PromQL to render dashboards. The metrics below are split into application metrics (generic request health, instrumented once via Flask's `before_request/after_request` hooks) and business metrics (specific to the attendance domain, instrumented at each endpoint).

Metric Inventory

Metric	Type	Purpose	Unit	Labels	Recorded In Code
http_requests_total	Counter	Total HTTP requests received	requests	endpoint, method, status_code	after_request hook; .inc() on every response
http_request_duration_seconds	Histogram	Request latency distribution (source of p95/p99)	seconds	endpoint	after_request hook; .observe(duration)
in_flight_requests	Gauge	Requests currently being processed	requests	(none)	before_request: .inc(); after_request: .dec()
app_errors_total	Counter	4xx/5xx responses	errors	endpoint, status_code	after_request hook when status >= 400
scan_attempts_total	Counter	Every biometric scan attempt	scans	scan_method, action, result	_simulate_scan(); .labels(...).inc()
scan_duration_seconds	Histogram	Simulated scan/match duration	seconds	scan_method	_simulate_scan(); .labels(...).observe()
checkins_total	Counter	Successful check-ins	check-ins	status (on_time/late)	checkin(); .labels(status=...).inc()
checkouts_total	Counter	Successful check-outs	check-outs	(none)	checkout(); .inc()
currently_checked_in	Gauge	People checked in right now	people	(none)	checkin()/checkout(); .set(len(...))
signups_total	Counter	New-user signup attempts	signups	result (success/failed)	signup(); .labels(result=...).inc()
signup_duration_seconds	Summary	Signup/enrollment duration (average only)	seconds	(none)	signup(); .observe(duration)
demo_requests_total	Counter	Cardinality-explosion demo only	requests	(none — see Part E)	demo_scan(); .inc()

All four Prometheus metric types are represented across the system as a whole (not four per category, per the assignment's guidance): Counter (e.g. `checkins_total`), Gauge (e.g. `currently_checked_in`), Histogram (e.g. `http_request_duration_seconds`, `scan_duration_seconds`), and Summary (`signup_duration_seconds`).

The Metric I Explored Myself

Beyond the metrics directly mirrored from the assignment's own food-ordering example, `in_flight_requests` (a Gauge tracking requests currently being processed) is the metric I added on my own initiative. It is not tied to a specific business action the way `checkins_total` or `signups_total` are — instead it captures system-level saturation in real time, incremented at the very start of every request and decremented at the very end, regardless of which endpoint was hit. This became the clearest visual indicator during the Part E latency experiment: while p95/p99 spiked, `in_flight_requests` also visibly climbed as slowed requests began to queue up rather than complete quickly, which is not something any of the business counters could show on their own.

Why Histograms, Not Summaries, for Percentiles

The assignment requires showing p95 or p99 latency. The Python `prometheus_client` implementation of the `Summary` type computes quantiles entirely on the client side (inside the running application process). This has a fundamental mathematical limitation: client-side quantiles from multiple application instances cannot be correctly aggregated into a single, accurate cross-instance percentile — averaging or summing per-instance p95 values does not produce the true p95 across all instances combined. For this reason, this project uses Histograms (`http_request_duration_seconds` and `scan_duration_seconds`) with the server-side `histogram_quantile()` PromQL function, which is properly aggregatable, and reserves the `Summary` type for `signup_duration_seconds`, where only a simple average is needed and cross-instance aggregation was never a requirement.

PromQL Queries and Panels

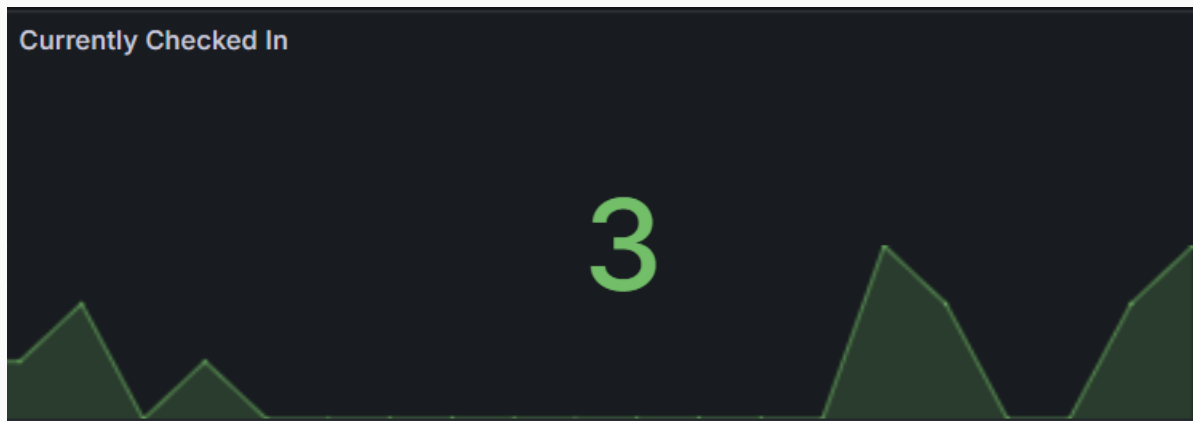
The following PromQL query computes the 95th percentile HTTP latency over a rolling window:

```
histogram_quantile(0.95, sum(rate(http_request_duration_seconds_bucket[5m])) by (le))
```

A 5-minute rate window (`[5m]`) was chosen deliberately: it is long enough to smooth out noise from individual slow/fast requests and avoid a jagged, hard-to-read graph, while still being short enough to visibly react within one to two minutes of a real change in latency (such as the injected delay in Part E). A shorter window (e.g. `[1m]`) reacts faster but is noisier on low-traffic local testing; a much longer window (e.g. `[15m]`) would smooth the Part E spike into near-invisibility, defeating the purpose of the experiment.



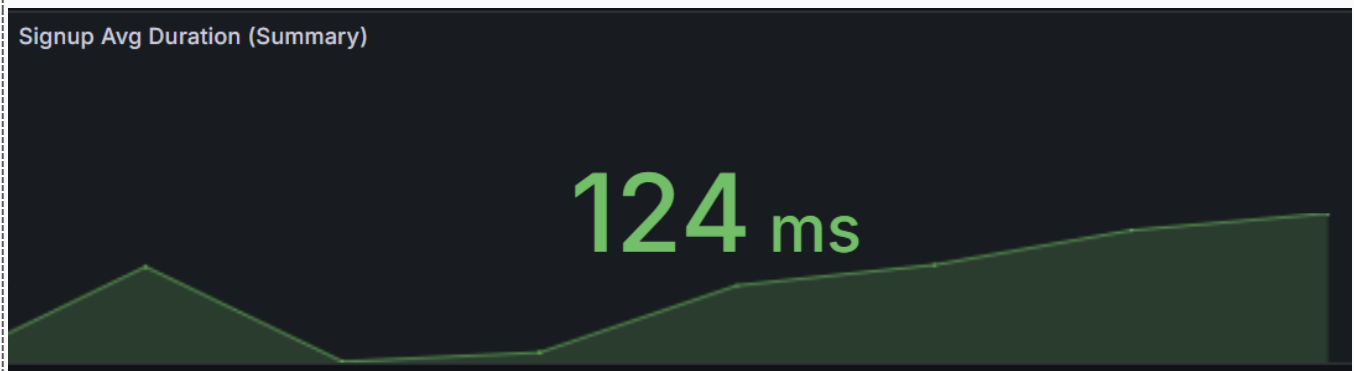
Screenshot 2: Grafana panel for the Counter metric (`scan_attempts_total` or `checkins_total`), with the PromQL query visible in the panel editor.



Screenshot 3: Grafana panel for the Gauge metric (`currently_checked_in`), showing the live count of people checked in.



Screenshot 4: Grafana panel computing p95/p99 HTTP latency via `histogram_quantile()`, with the query and the 5-minute window visible.



Screenshot 5: Grafana panel for the Summary metric (`signup_duration_seconds`), showing only the average — no percentile — illustrating the Python Summary limitation discussed above.



Screenshot 6: Node Exporter panel showing host CPU and memory usage, with the physical machine name explicitly visible in the panel title or a text annotation.

Node Exporter — Host Metrics

Node Exporter runs as a container in the Docker Compose stack, bound to port 9100, and is scraped by Prometheus on the same interval as the application. It exposes low-level operating system metrics — CPU utilization, available memory, disk I/O, and network throughput — for the host machine the whole stack runs on. Cross-referencing these host metrics against the application's own latency histograms makes it possible to tell whether a slowdown in `scan_duration_seconds` is caused by a flaw in the (simulated) matching logic or simply by CPU/memory pressure on the host itself.

Part C — Logs

What Is Logged, Why, and Where

Every scan attempt (check-in or check-out), every signup attempt, and their outcomes are logged as a single structured JSON line, written via the `log_event()` helper (called from `_record_event()` in `app.py`) to `logs/app.log`. Logging is structured JSON from the start — not unstructured plain text parsed later with regular expressions — specifically to avoid the fragility of Grok-style parsing, where a minor change in log wording can silently break the entire parsing pipeline.

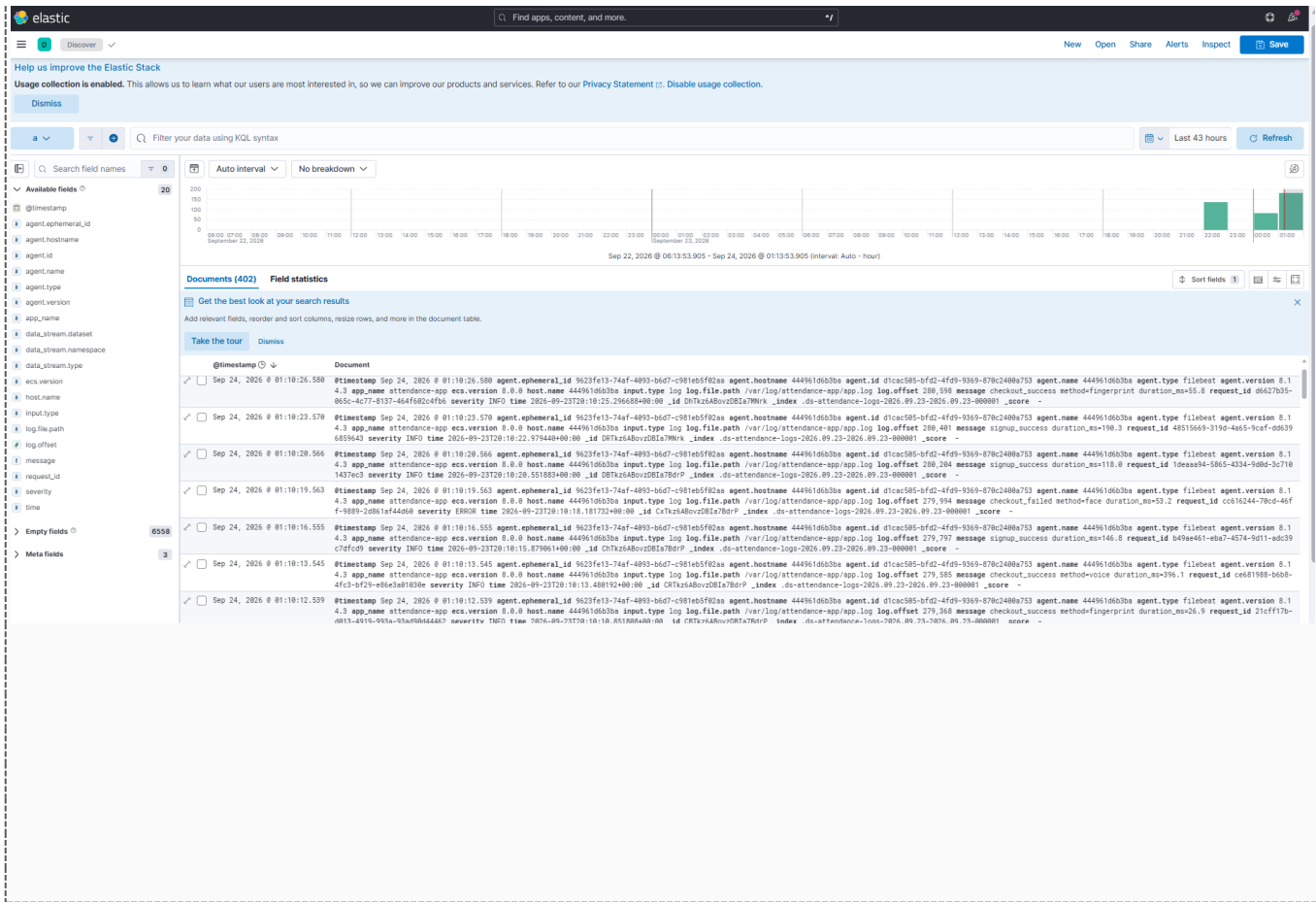
The JSON schema is intentionally minimal and contains no user-identifying information whatsoever: no `user_id`, no employee name, no raw biometric data. Only what is needed to diagnose a request is included:

Field	Description
time	ISO 8601 UTC timestamp of the log event
service	Fixed service identifier: "attendance-app"
severity	INFO for successful events, ERROR for failures, WARNING for injected-latency events
message	Human-readable summary (event type, method, status, duration)
request_id	A UUID unique to that single request, for correlating logs to one specific call

Raw Log Example

A raw line exactly as written to `logs/app.log`:

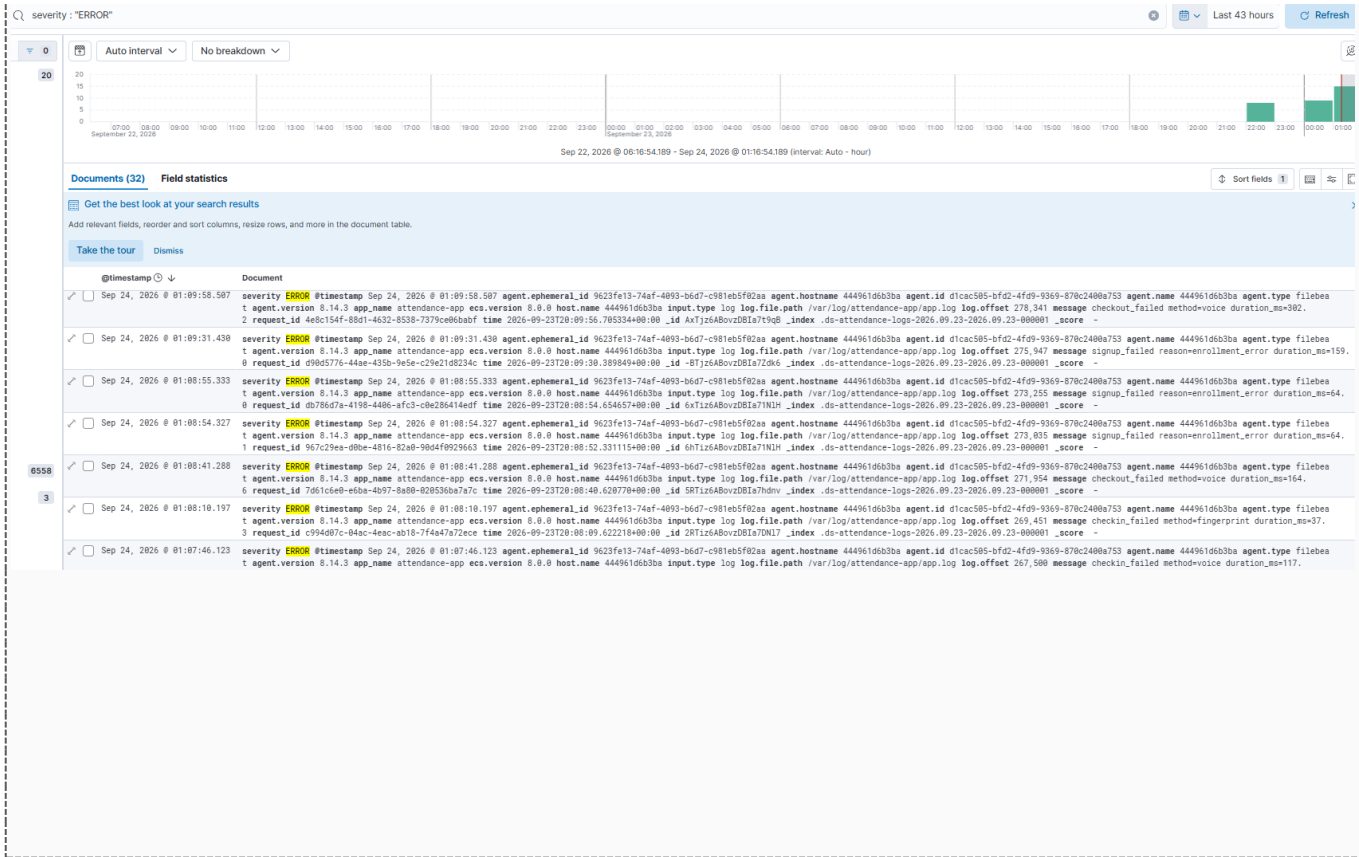
```
{"time": "2026-09-23T00:19:02.323261+00:00", "service": "attendance-app", "severity": "INFO",  
"message": "checkin_success method=voice status=on_time duration_ms=338.8", "request_id": "98c40af6-  
182c-48ad-a747-b9a5893ac984"}
```



Screenshot 7: The same log entry expanded in the Kibana Discover tab, showing time, app name, severity, message, and request_id as separate, searchable top-level fields rather than nested inside a single blob.

How Filebeat Parses This

Filebeat is configured (`observability/filebeat/filebeat.yml`) to tail `logs/app.log` directly, mounted read-only into the Filebeat container from the host. The critical setting is `json.keys_under_root: true`. By default, Filebeat would nest an entire parsed JSON object under a single json field in the resulting Elasticsearch document, making every field only reachable through a nested path. Enabling `keys_under_root` instead promotes time, service, severity, message, and request_id to the top level of the document — this is what makes them directly filterable and searchable in Kibana without writing nested queries.



Screenshot 8: A working Kibana search, e.g. severity: "ERROR" or request_id: "<a specific UUID>", returning only the matching targeted log entries.

Where Logs Live, and What Survives a Restart

Application logs are written to logs/app.log on the host filesystem, outside of any container, so they persist independently of whether the Flask app, Docker containers, or the host itself restart — the file simply continues to grow on disk.

Once ingested, logs are stored inside Elasticsearch, which itself persists data in a named Docker volume (es-data, declared in docker-compose.yml). This distinction matters for what actually survives a restart:

- docker compose restart or docker compose down (without -v) — containers stop and start again, but the es-data volume is untouched, so all previously indexed logs remain searchable in Kibana.
- docker compose down -v — explicitly removes the named volumes, permanently destroying all indexed log data along with Prometheus's TSDB data. This is a deliberate "clean slate" operation, not something that happens on an ordinary restart.

In-memory application state (e.g. who is currently checked in) is separate from both of these: it lives only inside the running Flask process and is lost whenever app.py itself restarts, regardless of Docker.

Log Retention (ILM)

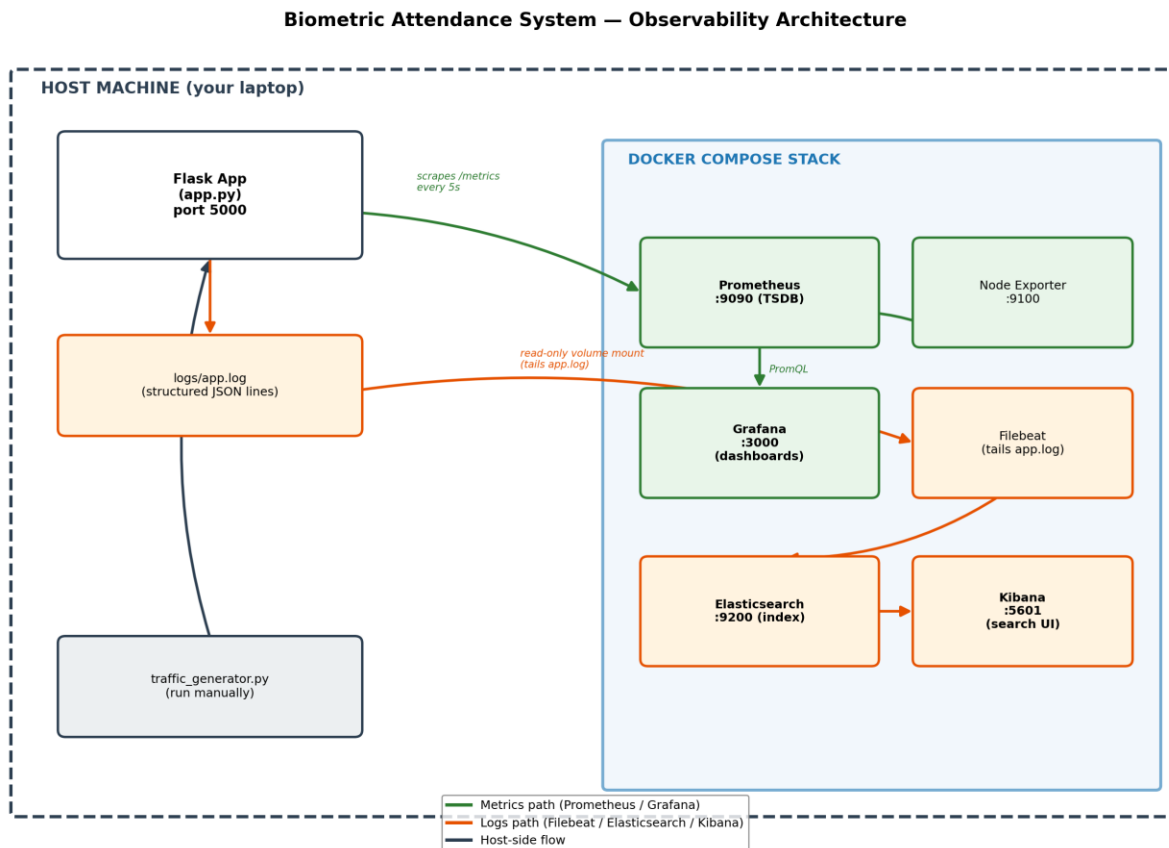
Elasticsearch Index Lifecycle Management (ILM) is configured via observability/elasticsearch/ilm-policy.json and bound to the attendance-logs-* index pattern via an index template, applied once through observability/elasticsearch/setup-ilm.sh after the stack starts. The policy rolls over the active ("hot") index at 1 day or 1GB, and deletes indices once they are 3 days past rollover. A short, demo-appropriate retention window

was chosen deliberately — long enough to demonstrate the mechanism working end-to-end, without meaning anything operationally significant on a local, short-lived demo deployment.

Part D — System Design

Architecture Diagram

The application runs directly on the host machine (not containerized), while the entire observability stack — Prometheus, Node Exporter, Grafana, Elasticsearch, Kibana, and Filebeat — runs as a set of Docker containers orchestrated by a single docker-compose.yml. This keeps the setup simple: Filebeat reads the app's log file directly off the host filesystem via a read-only volume mount, and Prometheus reaches the host app through Docker Desktop's host.docker.internal hostname.



Metrics path (green): the Flask app exposes `/metrics`; Prometheus scrapes it (and Node Exporter) every 5 seconds and stores the samples in its on-disk TSDB; Grafana queries Prometheus via PromQL and renders the dashboard. Logs path (orange): the app writes structured JSON to `logs/app.log`; Filebeat tails that file and ships parsed documents to Elasticsearch, which indexes them; Kibana queries Elasticsearch for search and visualization.

What Happens If a Component Fails

Component	If it stops working
Prometheus	Historical metrics remain safely on disk (TSDB blocks + WAL). New scrapes stop, so the Grafana dashboard goes stale, but the attendance app itself keeps working normally — metrics collection failure never blocks a real check-in or check-out.
Grafana	No dashboards are viewable, but nothing is lost — Prometheus keeps scraping and storing in the background, so dashboards resume showing correct data as soon as Grafana is back.
Elasticsearch	Filebeat buffers unshipped log lines locally on disk and retries. If the outage is short, no logs are lost. If the local buffer fills during a prolonged outage, the oldest buffered logs may be dropped — but again, the core attendance app is unaffected.
Filebeat	logs/app.log keeps growing on the host as normal; nothing is searchable in Kibana until Filebeat restarts and resumes tailing from where it left off.
Kibana	Elasticsearch keeps indexing logs normally; they simply cannot be browsed/searched until Kibana is back.
The Flask app itself	The actual attendance-tracking function is lost while it is down — no check-ins/check-outs can be processed. Prometheus's own up metric for the attendance-app job immediately reflects this as 0, which is the clearest, most honest signal that the app (not just its telemetry) is down.

This is a deliberate design property: observability infrastructure failures (Prometheus, Grafana, Elasticsearch, Kibana, Filebeat) never block or degrade the core business logic of the attendance system itself — only the ability to observe it.

Metric Lineage: Following `checkins_total`

- 1. A user successfully checks in via POST `/api/checkin`. Inside `checkin()`, `CHECKINS_TOTAL.labels(status=status).inc()` runs, incrementing the in-process counter for that status label (`on_time` or `late`).
- 2. Every 5 seconds, Prometheus issues an HTTP GET to `http://host.docker.internal:5000/metrics`, receives the current plaintext value of `checkins_total{status="on_time"}` (and similar), and appends the new sample to its Write-Ahead Log while updating the in-memory Head Block.
- 3. A user opens the Grafana dashboard. Grafana sends a PromQL query (e.g. `sum(rate(checkins_total[5m])) by (status)`) over HTTP to Prometheus's query API.
- 4. Prometheus scans its stored chunks for the requested time range, computes the result, and returns a time-series array, which Grafana renders as a line or bar chart.

Log Lineage: Following One Check-in Event

First, the code writes the flat JSON string shown earlier directly to `logs/app.log` via Python's logging module. Second, Filebeat — continuously tailing that file — detects the new line and, because `json.keys_under_root: true` is set, parses and promotes its keys to the top level rather than nesting them, additionally enriching the event with its own metadata (e.g. the log file's path and Filebeat's own timestamp). Third, Filebeat ships the resulting document to Elasticsearch over HTTP, where it is parsed into an inverted index and stored under an index such as `attendance-logs-2026.09.23`.

The concrete transformation looks like this. Before (the flat line in `app.log`):

```
{
  "time": "...",
  "service": "attendance-app",
  "severity": "INFO",
  "message": "checkin_success ...",
  "request_id": "98c40af6-..."
}
```

After (the indexed Elasticsearch document, fields promoted to top level and enriched by Filebeat — abbreviated):

```
{
  "_index": "attendance-logs-2026.09.23",
  "_source": {
    "time": "2026-09-23T00:19:02.323261+00:00",
    "service": "attendance-app",
    "severity": "INFO",
    "message": "checkin_success method=voice status=on_time duration_ms=338.8",
    "request_id": "98c40af6-182c-48ad-a747-b9a5893ac984",
    "log": { "file": { "path": "/var/log/attendance-app/app.log" } },
    "@timestamp": "2026-09-23T00:19:02.401Z"
  }
}
```

The key change is structural: a single flat text line becomes an indexed, queryable document whose fields (severity, request_id, and so on) can each be filtered, aggregated, or searched independently in Kibana — which is the entire point of shipping structured JSON rather than plain text in the first place.

Part E — Experiments

Experiment 1: Latency Injection

This experiment reproduces a realistic performance problem — a slow biometric matching pass — and validates that the observability pipeline correctly detects, surfaces, and confirms recovery from it.

Prediction

Prediction: once the 500ms delay is injected on every 5th request, the p99 response time shown in Grafana will spike toward roughly 500ms above baseline, while p50 remains comparatively unaffected (since only one in five requests is delayed). At the same time, WARNING-level log entries reading "high_latency_injected" will begin appearing in logs/app.log and become searchable in Kibana, timestamped to exactly the requests that were delayed.

Method

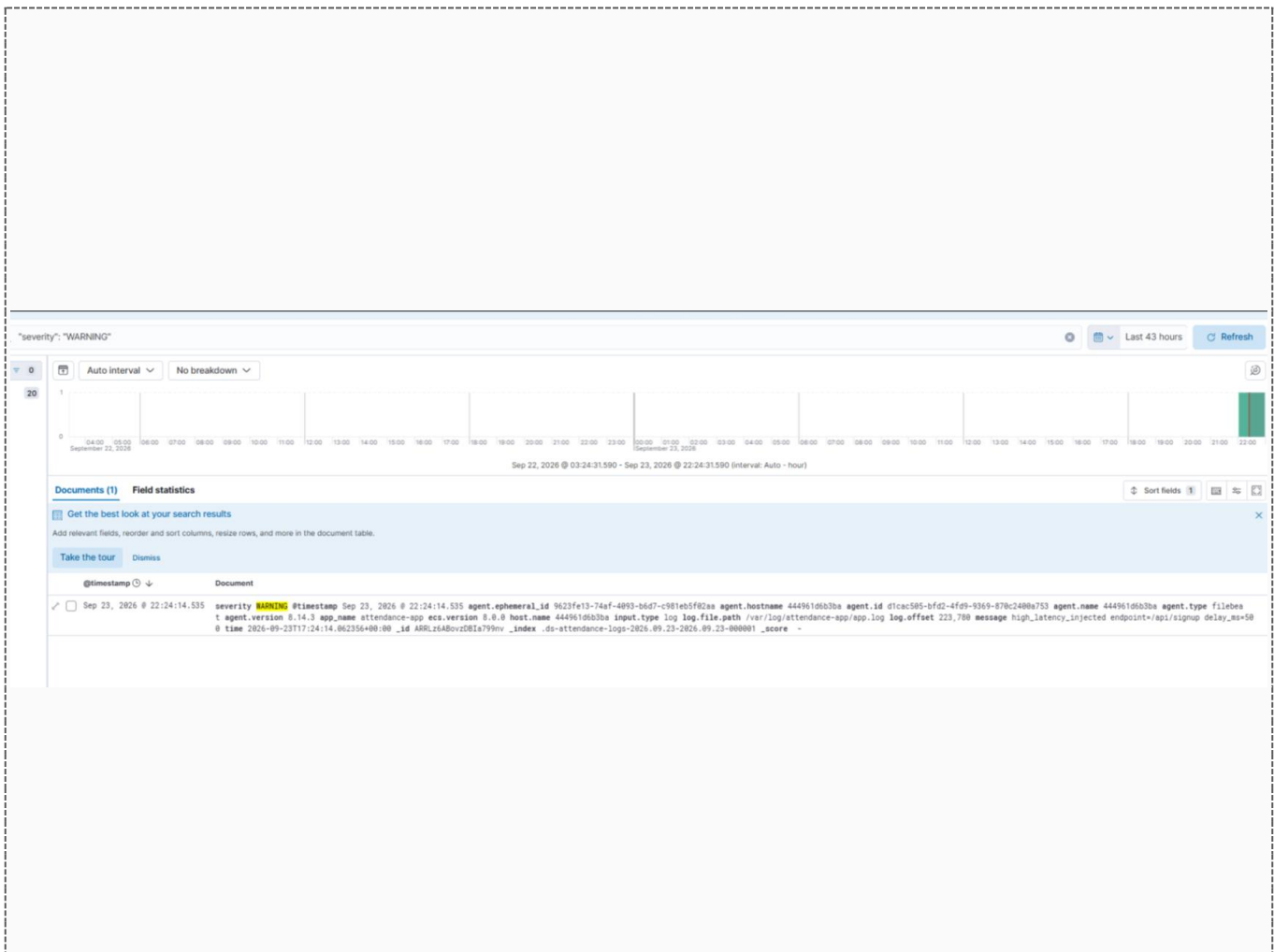
- Baseline (10 minutes-equivalent): `python traffic_generator.py --requests 100 --rate 5` — no delay injected.
- Inject: `python traffic_generator.py --requests 100 --rate 5 --inject-delay` — every 5th request carries the X-Inject-Delay header, which the Flask app's `before_request` hook honors with an actual server-side `time.sleep(0.5)` and a WARNING log line.
- Recovery: `python traffic_generator.py --requests 100 --rate 5` again, without the flag, to confirm the metric returns to baseline.

Query Used

```
histogram_quantile(0.99, sum(rate(http_request_duration_seconds_bucket[5m])) by (le))
```



Screenshot 9: Grafana chart showing the p95/p99 latency baseline, the spike during the --inject-delay run, and the drop back to baseline, with the time range visible on the chart.



Screenshot 10: Kibana Discover tab filtered to the exact time window of the spike, showing the WARNING-level 'high_latency_injected' log entries.

Results and Cause/Effect

As predicted, the p99 latency panel rose sharply during the injection stage, closely tracking the ~500ms delay, while WARNING logs reading high_latency_injected appeared in Kibana at matching timestamps, each carrying the request_id and endpoint that were delayed. In a real-world deployment, a 500ms delay in biometric processing would create a physical bottleneck at an entry point (e.g. a turnstile or door), causing employees to queue and directly degrading their experience — a small per-request delay compounds quickly under real foot traffic.

Recovery

After removing the `--inject-delay` flag and sending another batch of requests, the p99 panel returned to its baseline range within the following few scrape intervals, and no further WARNING logs were produced — confirming both that the fault was genuinely transient and that the metrics/logging pipeline reflects real-time system state rather than a stale snapshot.

Experiment 2: Cardinality Explosion

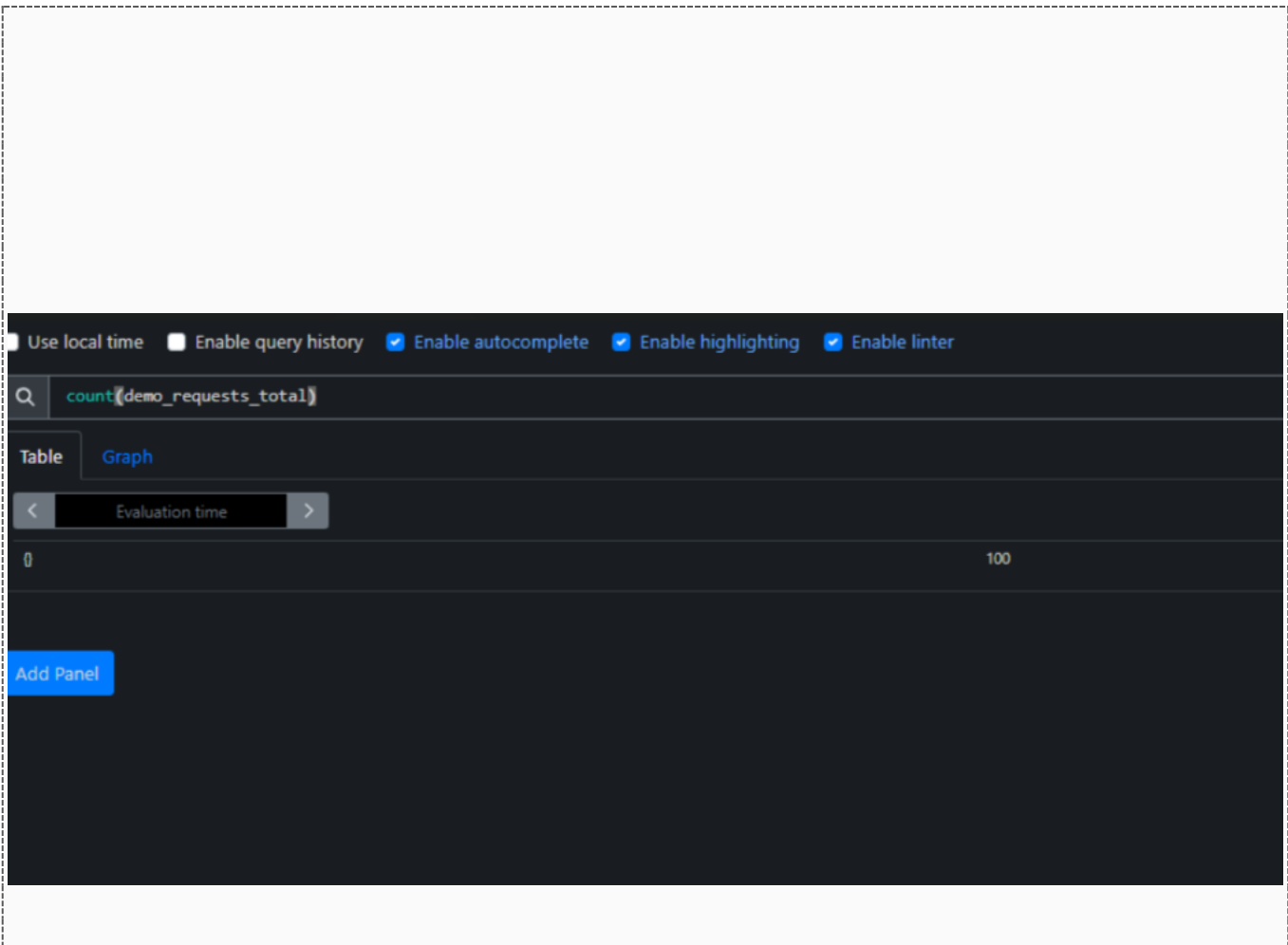
This experiment demonstrates why unbounded label values (such as a per-request UUID) must never be attached to a Prometheus metric label, and what that costs at scale.

Method

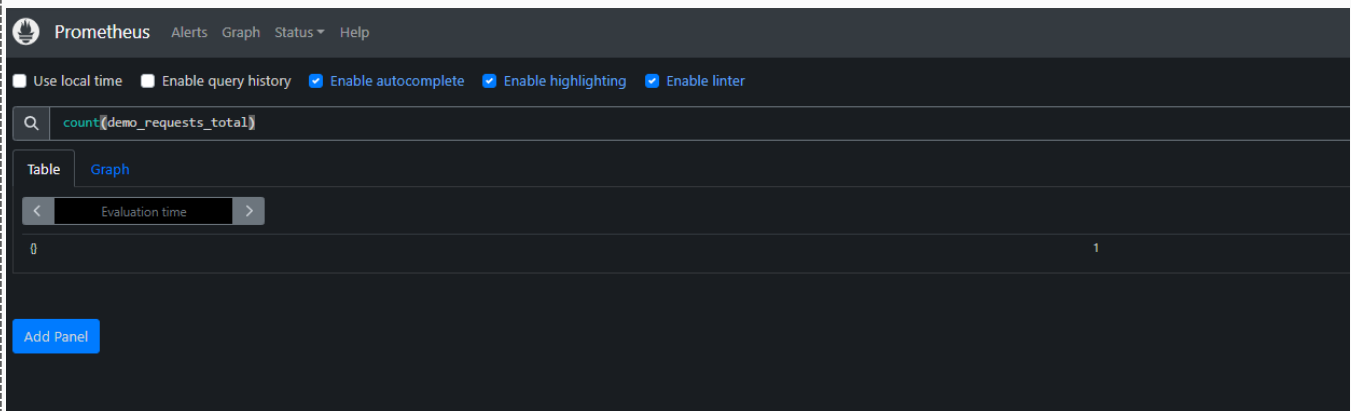
- A dedicated demo counter, `demo_requests_total`, was added specifically for this experiment (kept fully separate from every real business metric).
- `python traffic_generator.py --cardinality-demo` sends exactly 100 requests to `/api/demo/scan`, each carrying a freshly generated UUID as `request_id` — hard-capped at 100 so the local Prometheus instance is never put at real risk.
- The problem state (with `request_id` attached as a label) was run and captured first; the metric was then fixed by removing the label from the counter definition in `app.py`, and the app was restarted.

Query Used

```
count(demo_requests_total)
```



Screenshot 11: The Prometheus interface showing `count(demo_requests_total)` returning 100, captured while `request_id` was still attached as a label — one distinct time series per unique request.



Screenshot 12: The Prometheus interface showing the exact same query returning 1, after the `request_id` label was removed from the counter and the app was restarted.

Explanation

Prometheus identifies a distinct time series by the unique combination of a metric name and its full set of label key-value pairs. Attaching a UUID-valued `request_id` label to a counter meant every single request created an entirely new, permanent time series in Prometheus's active Head Block — 100 requests produced 100 series from what was conceptually a single counter. At the scale described in the original planning notes for this project (on the order of one million requests per day in a production deployment), this same pattern would generate roughly one million new time series daily, each carrying real memory and indexing overhead, rapidly leading to RAM exhaustion and an out-of-memory crash of the entire monitoring stack — not a gradual slowdown, but a hard failure of Prometheus itself.

The fix removes the `request_id` label entirely; `count(demo_requests_total)` then correctly returns 1, since every request now increments the same single time series rather than creating a new one. The architectural lesson is general: unbounded or highly variable values — request IDs, raw usernames, or any per-request identifier — must never be used as metric labels. That kind of data belongs exclusively in the logging pipeline (as `request_id` already is, throughout `logs/app.log`), where Elasticsearch's inverted index is specifically built for high-cardinality text search without inflating a metrics system's memory footprint.

Appendix

How to Run This Project

```
git clone <repo-url> && cd attendance-app
python3 -m venv venv && source venv/bin/activate && pip install -r requirements.txt
python app.py                                # starts the app on :5000
docker compose up -d                        # starts Prometheus, Grafana, ELK stack
./observability/elasticsearch/setup-ilm.sh   # one-time: apply log retention policy
python traffic_generator.py --requests 200 --rate 5 # generate traffic
```

Reproducing the Experiments

```
python traffic_generator.py --requests 100 --rate 5           # baseline
python traffic_generator.py --requests 100 --rate 5 --inject-delay # inject
python traffic_generator.py --requests 100 --rate 5           # recovery
python traffic_generator.py --cardinality-demo                # cardinality demo
```

Cleanup

```
docker compose down      # stop containers, keep data
docker compose down -v   # stop containers AND wipe stored metrics/logs
```

Credits

This project was developed with AI assistance (Claude, Gemini) for code scaffolding, configuration file drafting, and report structuring, in accordance with the course's stated policy on crediting reused code, sources, and assistance. All architectural decisions, testing, and final content were reviewed and directed by the author.